

Титульный лист

Содержание

Введение.....	3
Глава 1 Теоретическая основа автокорреляции и связь с многочленами Литтлвуда	5
1.1 Многочлены Литтлвуда. Автокорреляция	5
1.2 Плоские многочлены Литтлвуда	7
1.3 Многочлены Шапиро.....	8
Глава 2 Алгоритмы вычисления автокорреляции и построения многочленов Шапиро.....	9
2.1 Расчёт автокорреляции	9
2.2 Алгоритм построения многочленов Шапиро.	10
Глава 3 Реализация алгоритмов и анализ полученных результатов.....	12
3.1 Построение многочленов Шапиро и вычисление автокорреляции на языке C++	12
Autocorrelation.....	12
LittlewoodPolynomial.....	13
Tester	14
3.2 Тест вычисления автокорреляции	16
3.3 Проверка свойств многочленов Шапиро	17
3.4 Имитационное моделирование и статистический анализ эффективности последовательностей Шапиро	18
Заключение	22
Список литературы	23

Введение

Последовательности, состоящие из чисел ± 1 , применяются в радиолокации и цифровой связи, где требуется передавать или зондировать сигналы, хорошо различимые на фоне шумов. Основное требование к таким последовательностям – малая автокорреляция при всех ненулевых сдвигах, для того чтобы точно определять задержки сигнала.

Существует класс многочленов – многочлены Литтлвуда, коэффициенты которых принимают значения ± 1 . Коэффициенты таких многочленов можно рассматривать как последовательности применимые в задачах радиолокации и цифровой связи. В 2019 году была доказана теорема о существовании так называемых плоских многочленов Литтлвуда, значения которых на единичной окружности изменяются незначительно. Этот результат означает, что последовательности с близкими к оптимальным корреляционным свойствам существуют для любых длин, а значит, теоретически всегда можно построить сигнал с хорошими характеристиками различимости и устойчивости к шумам.

На практике длинные последовательности из ± 1 применяются, например, в системах радиолокации с синтетической апертурой и в современных цифровых средствах связи. В таких системах формируются зондирующие сигналы длиной до миллионов отсчётов, где последовательности из ± 1 служат кодом для фазовой модуляции импульсов.

Количество всех возможных бинарных последовательностей длины n равно 2^n , поэтому даже для последовательности длины 64 полный перебор становится невозможным. Это делает актуальной задачу разработки эффективных алгоритмов генерации и проверки последовательностей с нужными свойствами.

Вычисление автокорреляции – ключевая операция при оценке качества последовательности. В практических радиолокационных комплексах она используется для анализа откликов длинных сигналов, оптимизации кодов и моделирования сигналов с малыми боковыми лепестками корреляционной функции. Наивный алгоритм имеет квадратичную временную сложность, что

делает его непрактичным для длинных последовательностей. Применение алгоритма на основе быстрого преобразования Фурье (FFT) позволяет снизить сложность до $O(n \log n)$ и использовать этот метод на практике при обработке сигналов длиной из миллионов отсчётов.

В данной работе исследованы алгоритмы построения и анализа последовательностей, основанных на многочленах Литтлвуда, включая последовательности Шапиро. Реализован метод вычисления автокорреляции на основе FFT, а также наивный, для сравнения. Проведён анализ их временной сложности и производительности на смоделированном примере.

Глава 1 Теоретическая основа автокорреляции и связь с многочленами Литтлвуда

1.1 Многочлены Литтлвуда. Автокорреляция

Пусть есть последовательность $a = (a_0, a_1, \dots, a_{n-1}), a_i \in \{\pm 1\}$

Её автокорреляция определяется как функция сдвига. $R = (R_0, R_1 \dots R_{n-1})$

$$R_\tau(a) = \sum_{i=0}^{n-\tau-1} a_i a_{i+\tau}, \tau = 0, 1, 2 \dots n-1$$

Если для всех $\tau \neq 0$ R_τ малы, то последовательность имеет низкую автокорреляцию.

Алгоритм вычисления автокорреляции подразумевает попарное произведение всевозможных комбинаций элементов последовательности, т.е. имеет временную сложность $O(n^2)$. Можно оптимизировать временные затраты используя свойства свёртки и быстрого преобразования Фурье.

Свёрткой двух последовательностей $a = (a_0, a_1, \dots, a_{n-1})$ и $b = (b_0, b_1, \dots, b_{m-1})$ называется последовательность $a * b = c = (c_0, c_1, \dots, c_{n+m-1})$ элементы которой вычисляются как сумма произведений элементов двух последовательностей сдвинутых друг относительно друга.

$$c_k = \sum_{i=0}^{n-1} a_i b_{k-i} \quad k = 0, 1, \dots, n+m-2$$

Подразумевается, что при $k-i < 0$ элементы соответствующей последовательности равны нулю. Так как последовательностям можно сопоставить многочлены $A(z) = a_0 + a_1 z + \dots + a_{n-1} z^{n-1}$ $B(z) = b_0 + b_1 z + \dots + b_{m-1} z^{m-1}$, и произведение $A(z)B(z) = C(z) = c_0 + c_1 z + \dots + c_{n+m-1} z^{n+m-1}$, где

$$c_k = \sum_{i=0}^{n-1} a_i b_{k-i}$$

Получаем, что формула для свёртки последовательностей в точности совпадает с формулой для вычисления коэффициентов произведения

многочленов. Значит, автокорреляцию можно вычислить, используя произведение многочленов.

Для вычисления автокорреляции необходимо рассмотреть свёртку двух последовательностей, где одна из них перевёрнута. Это следует из формулы свёртки: индекс второго множителя вычитается из индекса свёртки. Алгоритм упрощается, так как автокорреляция считается для одной последовательности, то есть для вычисления потребуется произведение многочлена самого на себя. В спектральной области переворот последовательности соответствует комплексному сопряжению результата FFT, поэтому автокорреляцию можно получить следующим образом.

$$R_{\tau}(a) = \sum_{i=0}^{n-1} a_i a_{i+\tau} = (a * \hat{a})_{\tau}, \text{ где } \hat{a} = (a_{n-1}, a_{n-2}, \dots, a_0)$$

$$R(a) = (a * \hat{a}) = F^{-1}(F(A) \cdot \overline{F(A)}) = F^{-1}(|F(A)|^2)$$

В формуле выше подразумевается поэлементное умножение значений после применения быстрого преобразования Фурье и применение обратного преобразования Фурье к последовательности из квадратов элементов получившейся последовательности значений.

Быстрое преобразование Фурье позволяет выполнять алгоритм вычисления автокорреляции за время $O(n \log n)$.

Задача заключается в том, чтобы находить такие последовательности, которые имеют наименьшую автокорреляцию для всех τ . Рассмотрим один из вариантов поиска таких последовательностей.

Существует класс многочленов – многочлены Литтлвуда, коэффициенты которых принимают значения ± 1 .

$$P(z) = \sum_{i=0}^{n-1} a_i z^i, \text{ где } a_i \in \{\pm 1\}$$

Такие многочлены порождаются последовательностями, о которых было сказано выше. Они имеют множество интересных свойств, в частности, 6 лет назад доказано существование плоских многочленов Литтлвуда.[3]

1.2 Плоские многочлены Литтлвуда

Многочлены Литтлвуда называются плоскими, если $\exists c_1, c_2 > 0 \forall n : c_1\sqrt{n} \leq P_n(z) \leq c_2\sqrt{n} : |z| = 1$. Это означает, что теоретически для любой степени n можно найти многочлен, который будет плоским, то есть его модуль на единичной окружности будет оставаться примерно равным $\sqrt{n}$.

Последовательности, состоящие из ± 1 порождают многочлены Литтлвуда. Рассмотрим квадрат модуля на единичной окружности.

$$|P(e^{i\theta})|^2 = P(e^{i\theta})\overline{P(e^{i\theta})} = \sum_{k=0}^{n-1} a_k e^{ik\theta} \sum_{j=0}^{n-1} a_j e^{-ij\theta} = \sum_{m=-(n-1)}^{n-1} c_m e^{im\theta}$$

$$c_m = \sum_{k=0}^{n-m-1} a_k a_{k+m}$$

Эта запись показывает, что коэффициенты при экспонентах в разложении квадрата модуля соответствуют значениям автокорреляции последовательности. Таким образом, квадрат модуля многочлена Литтлвуда на единичной окружности напрямую связан с автокорреляцией последовательности, порождающей этот многочлен:

$$|P(e^{i\varphi})|^2 = \sum_{k=-(n-1)}^{n-1} R_k e^{ik\varphi}$$

Все боковые автокорреляции, то есть автокорреляции со сдвигами малы из-за плоскости. Получаем, что плоские многочлены Литтлвуда порождают последовательности с лучшими радарными свойствами.

Существует несколько известных классов плоских многочленов Литтлвуда. Каждый из этих типов имеет свои уникальные свойства и применения в анализе сигналов и теории чисел. В данной работе рассматриваются многочлены Шапиро. Они обладают особыми спектральными характеристиками и лучшими свойствами с точки зрения минимизации автокорреляции и генерации псевдослучайных последовательностей.

1.3 Многочлены Шапиро

Многочлены Шапиро определяются рекурсивно по формулам

$$P_{n+1}(x) = P_n(x) + x^{2^n} Q_n(x)$$

$$Q_{n+1}(x) = P_n(x) - x^{2^n} Q_n(x)$$

$$P_0(x) = Q_0(x) = 1$$

$$P_1(x) = 1 + x, \quad P_2(x) = 1 + x + x^2 - x^3$$

$$P_3(x) = 1 + x + x^2 - x^3 + x^4 + x^5 - x^6 + x^7$$

$$Q_1(x) = 1 - x, \quad Q_2(x) = 1 + x - x^2 + x^3$$

$$Q_3(x) = 1 + x + x^2 - x^3 - x^4 - x^5 + x^6 - x^7$$

Из определения следует, что степень многочленов равна $N = 2^n - 1$.

Выведем оценку модуля многочлена на единичной окружности. Для этого рассмотрим сумму квадратов модулей многочленов P и Q на единичной окружности.

Пусть $D = \{z \in \mathbb{C} : |z| = 1\}$. Далее подразумеваем, что $z \in D$.

$$|P_{n+1}(z)|^2 + |Q_{n+1}(z)|^2$$

$$|P_n(z) + z^{2^n} Q_n(z)|^2 + |P_n(z) - z^{2^n} Q_n(z)|^2$$

$$[\text{используем свойство: } |a + b|^2 + |a - b|^2 = 2|a|^2 + 2|b|^2]$$

$$2(|P_n(z)|^2 + |z^{2^n} Q_n(z)|^2) = 2(|P_n(z)|^2 + |Q_n(z)|^2)$$

Покажем по индукции, что $|P_n(z)|^2 + |Q_n(z)|^2 = 2^{n+1}$

$$\text{База: } n = 0 \Rightarrow |P_0(z)|^2 + |Q_0(z)|^2 = 1 + 1 = 2 = 2^{0+1}$$

Предположим, что для $k \geq 0$ выполняется: $|P_k(z)|^2 + |Q_k(z)|^2 = 2^{k+1}$

$$\text{для } k + 1: |P_{k+1}(z)|^2 + |Q_{k+1}(z)|^2 =$$

$$2(|P_k(z)|^2 + |Q_k(z)|^2) = 2 \cdot 2^{k+1} = 2^{k+2}$$

Так как слагаемые неотрицательны:

$$|P_n(z)|^2 \leq |P_n(z)|^2 + |Q_n(z)|^2 \leq 2^{n+1} = 2N$$

Отсюда следует итоговая оценка $|P_n(z)| \leq \sqrt{2N}$.

Поэтому последовательности, образованные коэффициентами многочленов Шапиро, должны иметь хорошую автокорреляцию.

Глава 2 Алгоритмы вычисления автокорреляции и построения многочленов Шапиро

2.1 Расчёт автокорреляции

Пусть дана последовательность $(a_0, a_1, \dots, a_{n-1})$. Требуется вычислить её автокорреляцию. Интересна реализация, основанная на FFT, но для сравнения также опишем наивную. Наивный метод заключается в вычислении автокорреляции по определению: для каждого сдвига τ вычисляется сумма попарных произведений элементов последовательности и её сдвинутой копии. Такой подход требует выполнения двух вложенных циклов, внешний из которых перебирает сдвиги, а внутренний индексы элементов. Временная сложность данного метода составляет $O(n^2)$, поскольку для каждого из n сдвигов производится $n - \tau$ умножений и суммирований. Метод прост в реализации и удобен для коротких последовательностей, однако становится неэффективным при увеличении длины n из-за квадратичного роста времени вычислений.

Для оптимизации вычисления автокорреляции опишем метод, основанный на быстром преобразовании Фурье. Пусть n – длина исходной последовательности. Автокорреляция последовательности определяется коэффициентами произведения двух многочленов, соответствующих исходной последовательности и её перевёрнутой версии. Каждый из этих многочленов имеет степень $n - 1$, и их произведение является многочленом степени $2n - 2$, коэффициенты которого совпадают с автокорреляцией для всех сдвигов, как положительных, так и отрицательных.

Чтобы корректно вычислить произведение многочленов через FFT, необходимо определить значения многочлена в достаточном количестве точек на единичной окружности. Чтобы однозначно восстановить многочлен степени $2n - 2$ необходимо вычислить значения в $2n - 1$ точке. Для удобства деления последовательности пополам на каждом шаге алгоритма выбирается число m – ближайшая степень двойки, превышающая или равная $2n - 1$. Исходная последовательность и её перевёрнутая версия дополняются нулями до длины

m . После дополнения нулями выполняется прямое быстрое преобразование Фурье каждой последовательности. Эта операция имеет временную сложность $O(m \log m)$. Полученные спектры перемножаются поэлементно, что соответствует произведению многочленов в точках на единичной окружности. Эта операция имеет временную сложность $O(m)$. Затем выполняется обратное FFT, и в результате получаем коэффициенты произведения многочленов. Эти коэффициенты и являются значениями автокорреляции исходной последовательности для всех сдвигов.

В описанном алгоритме используется стандартная реализация FFT, которая требует дополнения последовательности до длины, равной ближайшей степени двойки, для корректного деления на половины на каждом шаге. Эта реализация имеет временную сложность $O(m \log m)$, где m – длина дополненной последовательности. Существуют более продвинутые реализации FFT, способные выполнять преобразование Фурье последовательности длины n со сложностью $O(n \log n)$. Такие реализации позволяют дополнительно ускорить вычисления автокорреляции и других операций, основанных на свёртках или произведениях многочленов.

2.2 Алгоритм построения многочленов Шапиро

Перепишем многочлен $P_{n+1}(x) = P_n(x) + x^{2^n} Q_n(x) = [P_{n-1}(x^2) + xP_{n-1}(-x^2)] + x^{2^n} [Q_{n-1}(x^2) + xQ_{n-1}(-x^2)] = [P_{n-1}(x^2) + x^{2^n} Q_{n-1}(x^2)] + x[P_{n-1}(-x^2) + x^{2^n} Q_{n-1}(-x^2)]$

Получаем: $P_{n+1}(x) = P_n(x^2) + xP_n(-x^2), n \geq 0$

Тогда коэффициент $a_r \in \{\pm 1\}$, стоящий перед r -ой степенью многочлена удовлетворяет рекуррентному соотношению $a_{2r} = a_r$, $a_{2r+1} = (-1)^r a_r$

Если записать $r = r_0 + 2r_1 + 4r_2 + \dots + 2^k r_k$ $k \geq 0$, где $r_i = 0, 1$. Тогда $a_r = (-1)^{r_0 r_1 + r_1 r_2 + \dots + r_{k-1} r_k}$ (Доказательство описано в [2])

То есть знак коэффициента определяется чётностью числа подряд идущих единичных битов в двоичной записи индекса r .

Для каждого коэффициента a_k заводим счётчик c_k пар подряд идущих единиц в двоичном разложении индекса k . Сравниваем результат применения «побитового и» к индексу k с числом 3, так как $3_{10} = 11_2$. Эта операция выделит два младших бита. Если результат операции равен 3 (то есть оба бита – единицы), значение счётчика увеличивается на единицу и убирается последний бит. Продолжаем те же действия пока $k \neq 0$. Затем вычисляем коэффициент многочлена $a_k = (-1)^{c_k}$. Продолжаем вычисления для остальных коэффициентов.

Алгоритм проходит по всем индексам $k = 0, 1 \dots n - 1$. и для каждого выполняется цикл, который делает количество итераций, равное числу бит в числе k . $O(\log_2 k) \leq O(\log_2 n)$.

Временная сложность:

$$T(n) = \sum_{k=0}^{n-1} O(\log_2 k) = O(n \log n)$$

Глава 3 Реализация алгоритмов и анализ полученных результатов

3.1 Построение многочленов Шапиро и вычисление автокорреляции на языке C++

Создадим три класса: *Autocorrelation* – для вычисления автокорреляции; *LittlewoodPolynomial* – для работы с многочленами Литтлвуда; *Tester* – для проведения тестов.

Autocorrelation

Класс *Autocorrelation* содержит две статические функции: *computeNaive* – вычисляет автокорреляцию попарным перебором всех элементов последовательности; *computeFFT* – вычисляет автокорреляцию используя быстрое преобразование Фурье.

Реализация наивного метода заключается в том, что для каждого сдвига k вычисляется сумма попарных произведений.

```
static std::vector<double> computeNaive(const std::vector<T>& seq) {
    if (seq.empty()) throw std::invalid_argument("Sequence must not be empty");

    size_t n = seq.size();
    std::vector<double> result(n, 0.0);

    for (size_t k = 0; k < n; ++k) {
        double sum = 0.0;
        for (size_t i = 0; i + k < n; ++i)
            sum += static_cast<double>(seq[i]) * static_cast<double>(seq[i + k]);
        result[k] = sum;
    }
    return result;
}
```

Листинг 1. Вычисление автокорреляции наивным способом

Метод с использованием FFT (*computeFFT*) основан на идее, заключающейся в том, что автокорреляция – это свёртка последовательности с самой собой, что в спектральной области превращается в произведение преобразований Фурье. Для реализации используется библиотека FFTW.

1. Последовательность дополняется нулями до длины $m \geq 2n + 1$.
2. Производится вычисление FFT последовательности, преобразуя действительные значения в комплексные спектры.

3. Рассчитывается модуль в квадрате комплексных чисел путём умножения спектра на его комплексно-сопряжённый.
4. Выполняется обратное FFT для получения автокорреляции в исходной области.
5. Результат делится на m для нормализации значений.

```
static std::vector<double> computeFFT(const std::vector<T>& seq) {
    if (seq.empty()) throw std::invalid_argument("Sequence must not be empty");
    size_t n = seq.size();
    size_t m = 1;
    while (m < 2 * (n + 1)) m <= 1;

    std::vector<double> in(m, 0.0);
    for (size_t i = 0; i < n; ++i) in[i] = static_cast<double>(seq[i]);
    fftw_complex* freq = (fftw_complex*) fftw_malloc(sizeof(fftw_complex) * (m/2 + 1));
    double* out = (double*) fftw_malloc(sizeof(double) * m);

    fftw_plan forward = fftw_plan_dft_r2c_1d(m, in.data(), freq, FFTW_ESTIMATE);
    fftw_plan backward = fftw_plan_dft_c2r_1d(m, freq, out, FFTW_ESTIMATE);
    fftw_execute(forward);

    for (size_t i = 0; i < m/2 + 1; ++i) {
        double re = freq[i][0];
        double im = freq[i][1];
        freq[i][0] = re * re + im * im;
        freq[i][1] = 0.0;
    }
    fftw_execute(backward);
    std::vector<double> result(n);
    for (size_t k = 0; k < n; ++k)
        result[k] = out[k] / m;
    fftw_destroy_plan(forward);
    fftw_destroy_plan(backward);
    fftw_free(freq);
    fftw_free(out);

    return result;
}
```

Листинг 2. Вычисление автокорреляции с помощью БПФ

LittlewoodPolynomial

Класс `LittlewoodPolynomial` предназначен для работы с многочленами Литтлвуда. Конструктор, принимающий степень d создаёт многочлен степени d со всеми коэффициентами равными 1. Конструктор принимающий `vector<int_8t>` создаёт многочлен по набору коэффициентов. Тип данных `int_8t` для хранения коэффициентов используется для экономии памяти. Перегружен оператор `[]` для взаимодействия с конкретным коэффициентом. Перегружен оператор `<<` для вывода многочлена в консоль. Основные методы:

`randomPolynomial` – генерирует случайным многочлен Литтлвуда;
`shapiroPolynomial` – генерирует многочлен Shapiro $P_n(x)$.

```
static LittlewoodPolynomial randomPolynomial(size_t degree) {
    LittlewoodPolynomial poly(degree);
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<int> dist(0, 1);

    for (size_t i = 0; i <= degree; ++i)
        poly[i] = dist(gen) ? 1 : -1;

    return poly;
}
```

Листинг 3. Генерация случайного многочлена Литтлвуда

```
static LittlewoodPolynomial shapiroPolynomialP(size_t degree) {

    if (((degree + 1) & degree) != 0)
        throw std::invalid_argument("Degree must be a power of 2 - 1");

    LittlewoodPolynomial poly(degree);
    for (size_t k = 0; k <= degree; ++k) {
        size_t x = k;
        int count = 0;
        while (x > 0) {
            if ((x & 3) == 3)
                count++;
            x >>= 1;
        }
        poly.coefficients[k] = (count % 2 == 0 ? 1 : -1);
    }
    return poly;
}
```

Листинг 4. Генерация $P_n(x)$ многочлена Шапиро.

Tester

Класс `Tester` создан для тестирования. В нём перегружен оператор `()` для выполнения тестов. Класс имеет поля: `int repeat` – количество повторений; `TimeUnit unit` — это вложенное перечисление с элементами `MILLISECONDS` и `SECONDS`. При инициализации объекта класса `Tester` можно выбрать количество повторений *repeat* и единицу измерения времени (по умолчанию – миллисекунды). При вызове функции через объект класса выполняется *repeat* повторений, после чего вычисляется среднее арифметическое всех запусков.

Пример использования:

```
Tester tester(100);
LittlewoodPolynomial poly = LittlewoodPolynomial::randomPolynomial(1000);
double average_fft = tester(&Autocorrelation<int8_t>::computeFFT, poly);

Вывод:
Run 1: 39.8779 ms
Run 2: 0.5193 ms
Run 3: 0.4457 ms
...
Run 100: 0.828 ms
Average time: 0.913513 ms
```

Листинг 5. Пример работы с Tester

Можно заметить, что первый запуск алгоритма `computeFFT` выполняется значительно дольше остальных. Это связано с тем, что библиотека FFTW при первом вызове создаёт и оптимизирует внутренний план выполнения FFT, включая выделение необходимых буферов и выбор оптимальных алгоритмов для данного размера последовательности. Эти накладные расходы возникают только один раз и не повторяются при последующих вызовах. Поэтому в тестах будем выполнять одиночный предварительный вызов `computeFFT` перед измерением времени, чтобы исключить влияние накладных расходов на инициализацию и получить корректное среднее время выполнения алгоритма.

Дополнительно Класс `Tester` дополнительно содержит шаблонную функцию `saveVectorToFile`, предназначенную для сохранения последовательностей (векторных данных) в текстовый файл. Эта функция полезна, так как часто возникает необходимость сохранять результаты генерации многочленов для анализа.

```
template<typename T>
static void saveVectorToFile(const std::vector<T>& vec, const std::string& filename =
"test_file.txt") {
    std::ofstream out(filename);
    if (!out.is_open()) throw std::runtime_error("Cannot open file " + filename);

    for (size_t i = 0; i < vec.size(); ++i) {
        out << static_cast<int>(vec[i]);
        if (i + 1 != vec.size())
            out << " ";
    }
    out << std::endl;
    out.close();
    std::cout << "Vector saved to " << filename << "\n";
}
```

Листинг 6. Реализация сохранения вектора в файл.

3.2 Тест вычисления автокорреляции

Функция *autocorrelationTest* выполняет временной анализ алгоритмов вычисления автокорреляции для случайных многочленов Литтлвуда различной степени. Для каждой степени от 10 до 500 включительно с шагом 10 генерируется случайный многочлен с помощью *LittlewoodPolynomial::randomPolynomial(n)*. Сначала выполняется наивное вычисление автокорреляции методом *computeNaive* через объект *tester* с 100 повторениями, затем выполняется предварительный вызов метода *computeFFT* и проводится 100 повторений вычисления автокорреляции с использованием FFT. В обоих случаях среднее время выполнения для каждой степени многочленов записывается в CSV-файл *autocorrelation_times.csv*. Ниже приведены графики зависимости средней времени выполнения алгоритмов от степени многочлена.

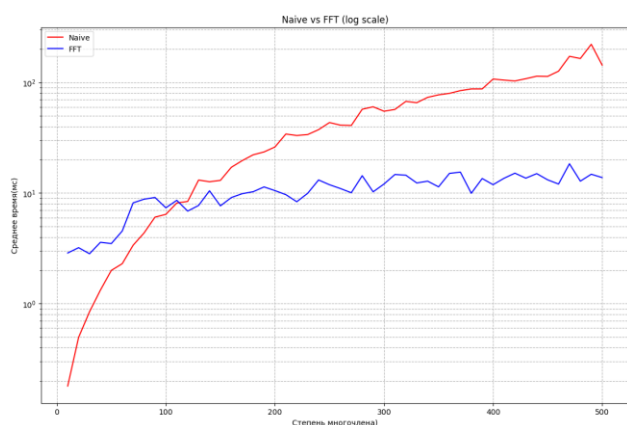


Рисунок 1. Сравнение времени выполнения алгоритмов вычисления автокорреляции(логарифмическая шкала)

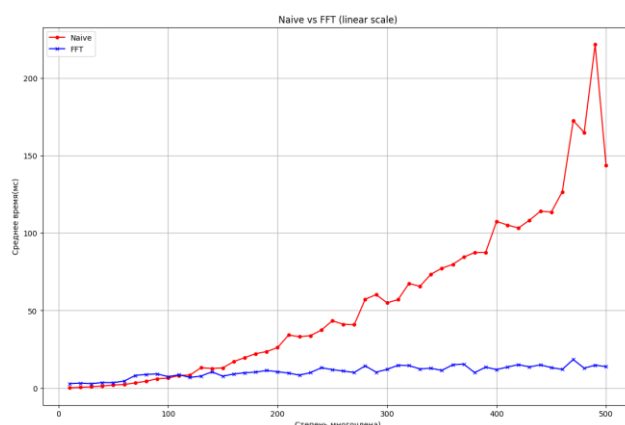


Рисунок 2. Сравнение времени выполнения алгоритмов вычисления автокорреляции(линейная шкала)

Алгоритм с использованием FFT выигрывает у наивного для многочленов 120 степени и больше, но учитывая малое время выполнения для меньших степеней можно сделать вывод о высокой эффективности FFT алгоритма.

Функция *autocorrelationFileTest* выполняет вычисление автокорреляции для многочлена Литтлвуда 1_000_000 степени, загруженного из файла *coeffs.txt*. Вычисляется автокорреляция с использованием FFT метода *computeFFT*, и результат сохраняется в файл *autocorrelation_result.txt*. Для измерения времени выполнения создаётся объект *tester*, который выполняет 10 вычислений и

вычисляет среднее время в миллисекундах. Результат, который получился 243мс (Тест проводился на системе с процессором AMD Ryzen 5 3500U, 6 ГБ оперативной памяти и операционной системой Windows 11)

3.3 Проверка свойств многочленов Шапиро

Функция *shapiroTest* выполняет построение и анализ многочленов Шапиро и случайных многочленов для последующего сравнения их свойств. Для многочленов степени 255 создаются случайный многочлен *r_poly* и многочлен Шапиро *s_poly* с использованием соответствующих методов класса *LittlewoodPolynomial*. Полученные коэффициенты сохраняются в файлы *random_polynimial.txt* и *shapiro_polynomial.txt* с помощью функции *saveVectorToFile*.

Далее данные загружаются в Python для анализа. Для каждого многочлена вычисляется боковая автокорреляция. Производится оценка модуля многочлена на единичной окружности с использованием FFT. Для многочлена Шапиро вычисляется теоретическая верхняя оценка нормы $c_2 = \sqrt{2(n)}$ (n – степень многочлена).

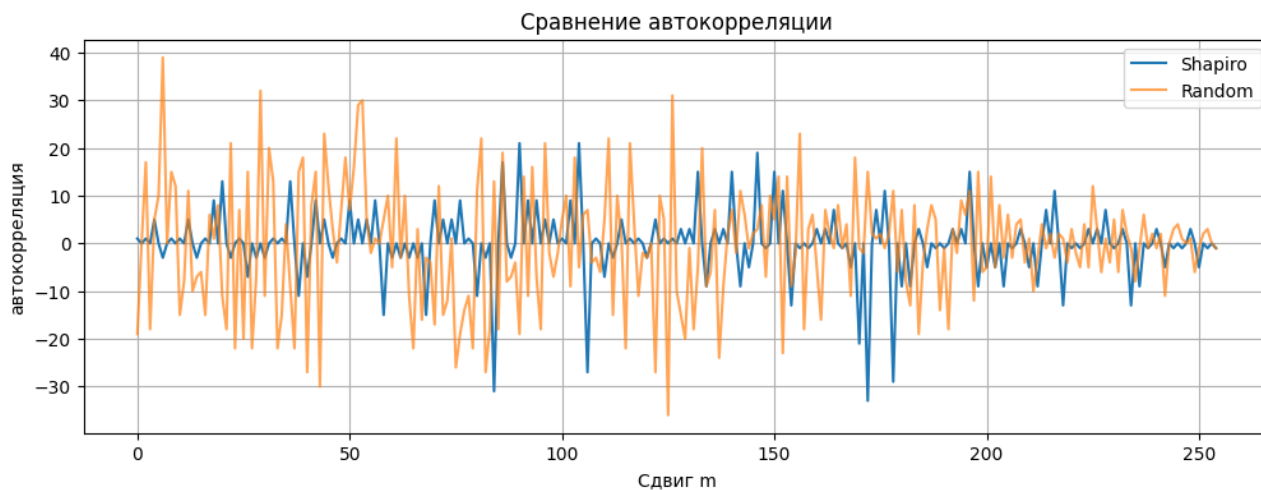


Рисунок 3. Сравнение автокорреляции случайного многочлена и многочлена Шапиро

Вычислено среднеквадратическое отклонение для обоих многочленов последовательностей. В случае случайной последовательности показатель составляет 12.327; для последовательности являющейся коэффициентами многочлена Шапиро 6.532.

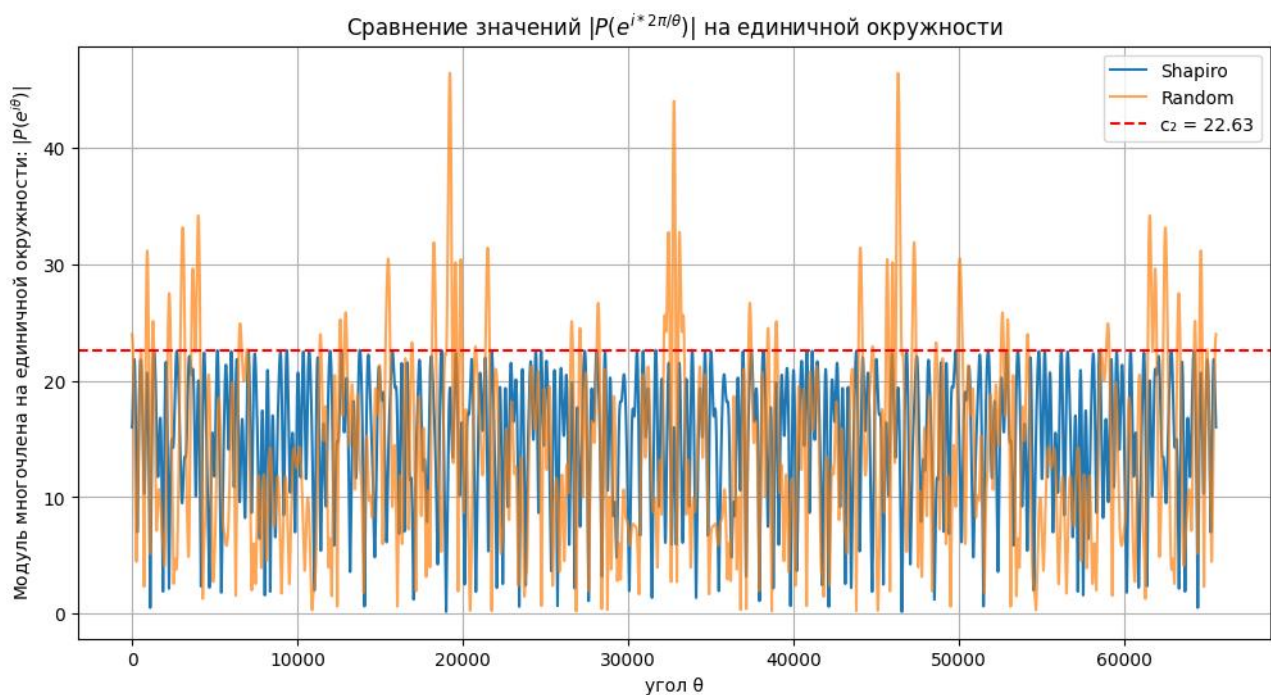


Рисунок 4. Сравнение значений многочленов на единичной окружности

Графики демонстрируют, что многочлены Шапиро обладают равномерным распределением автокорреляции и ограниченным ростом значений на единичной окружности, подтверждая их хорошие спектральные характеристики.

Максимальные значения боковой автокорреляции и модуля на единичной окружности вычисляются для каждого типа многочлена. Для многочлена Шапиро максимальное значение модуля не превышает $\sqrt{2(n)} = \sqrt{2 * 256} \approx 22.63$, что соответствует теоретической оценке.

3.4 Имитационное моделирование и статистический анализ эффективности последовательностей Шапиро

Для оценки практической эффективности последовательностей Шапиро было проведено имитационное моделирование процесса радиолокационного обнаружения, описанное в ноутбуке *radar_echo_simulation.ipynb*. Основная цель эксперимента заключалась в обнаружении целей, что является ключевой задачей при использовании рассматриваемых последовательностей.

В эксперименте проводился подбор оптимального относительного порога детектора для последовательностей Шапиро и случайных последовательностей.

Порог детектора определяет уровень корреляции, при котором сигнал считается откликом реальной цели, а не шумом. Оптимизация порога направлена на обеспечение баланса между количеством правильно обнаруженных целей и количеством ложных срабатываний.

Относительный порог детектора задается как доля от максимального значения корреляции между принятым сигналом и отправленной последовательностью:

$$T_{abs} = threshold_{rel} \cdot \max(|corr|),$$

где $\max(|corr|)$ – максимальное значение модуля корреляции для конкретного сигнала. Значение $threshold_{rel} = 0.45$ означает, что детектор срабатывает, если амплитуда корреляции превышает 45% от максимального значения корреляции.

Подбор оптимального порога осуществлялся следующим образом: задаётся набор значений $threshold_{rel}$ от 0.1 до 0.9 с шагом 0.05. Для каждого значения порога проводится серия симуляций с различными случайными параметрами канала, положением целей и уровнем шума. Для каждой симуляции вычислялись показатели: количество правильно обнаруженных целей *hits*, количество пропущенных целей *miss*, количество ложных срабатываний *false* и *F1-score*, представляющий гармоническое среднее точности и полноты. Для каждого значения порога определялось среднее значение *F1-score*. Значение порога, при котором *F1-score* достигало максимума было выбрано как оптимальное. В результате был выбран порог $threshold_{rel} = 0.45$.

После определения оптимального порога была проведена серия тестов для статистической оценки эффективности последовательностей. Для этого использовался выбранный порог. Выполнялось 2000 повторений с различными случайными параметрами канала, целей и уровнем шума. Для каждого прогона фиксировались показатели: количество правильных детекций, количество

пропущенных целей и количество ложных срабатываний. Результаты моделирования представлены в таблице.

Таблица 1. Результаты моделирования радиолокационного обнаружения

	Последовательность Шапиро	Случайная последовательность
Среднее число верных обнаружений	20.141	18.658
Среднее число пропусков	9.859	11.342
Среднее число ложных срабатываний	3.467	6.883

Пример одного теста с параметрами: 3 цели; частота 50МГц; 2 импульса; время между импульсами составляет 0.0008мс; уровень шума 1, что соответствует шуму с амплитудой, сопоставимой с сигналами отражённых целей; время до первой, второй и третьей целей составляют 0.002, 0.005 и 0.008мс соответственно.

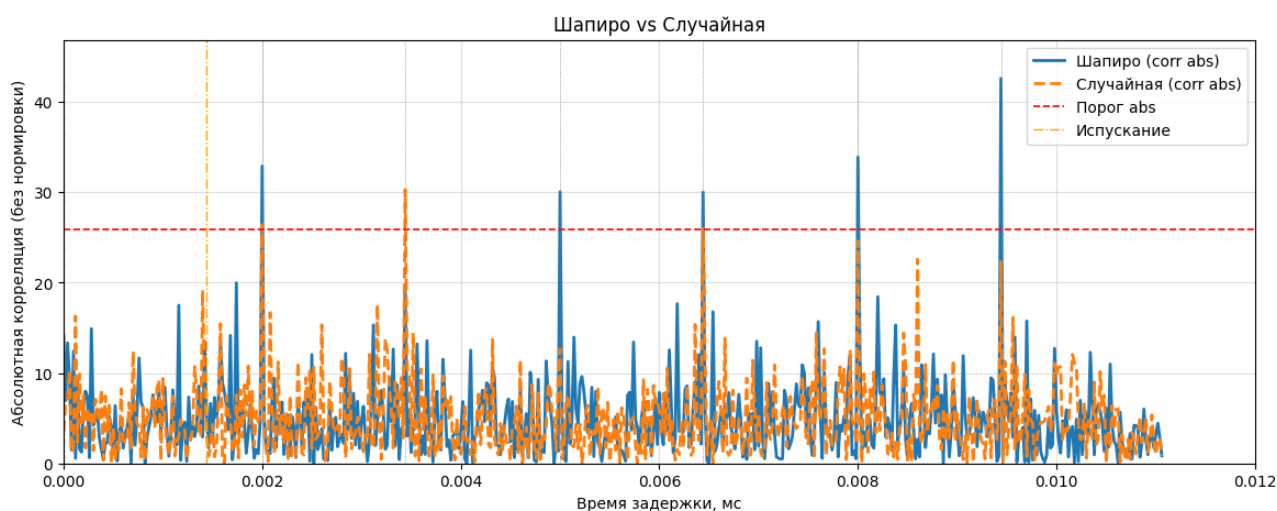


Рисунок 5. Демонстрация работы приёмника.

Таблица 2. Сводная таблица по целям

Номер цели	Время до цели(мс)	Обнаружения (последовательность Шапиро)	Обнаружения(Случайная последовательность)
1	0.002	1	2
2	0.005	2	1
3	0.008	2	0

Последовательность Шапиро имеет небольшое преимущество по количеству верных обнаружений и пропусков по сравнению со случайными последовательностями в большом эксперименте; наиболее выраженное различие наблюдается по числу ложных срабатываний. Последовательности Шапиро демонстрируют существенно меньший уровень ложных обнаружений, что соответствует их теоретически низкой боковой автокорреляции и подтверждает их эффективность в задачах радиолокационного обнаружения.

Более подробные параметры эксперимента и код реализации приведены в файле *radar_echo_simulation.ipynb*.

Заключение

В ходе работы изучены свойства многочленов Литтлвуда, описан алгоритм построения плоских многочленов Шапиро. Описан алгоритм для вычисления автокорреляции с использованием FFT. Приведена оценка сложностей алгоритмов. В результате проведённых тестов подтверждена теоретическая оценка модуля многочленов Шапиро на единичной окружности. Имитационное моделирование подтвердило эффективность использования последовательностей Шапиро в задачах радиолокационного обнаружения. Результаты тестирования и примеры реализации всех алгоритмов доступны на [github: asardinka](https://github.com/asardinka).

Список литературы

1. Кормен, Т. Х. Алгоритмы: построение и анализ / Т. Х. Кормен, Ч. Э. Лейзерсон, Р. Л. Ривест, К. Штайн. — М.: Вильямс, 2011. — 1312 с.
2. Roelfszema, M. Littlewood polynomials / M. Roelfszema. — Groningen: University of Groningen, 2015. — Bachelor's thesis.
3. Erdélyi T. Improved results on the oscillation of the modulus of the Rudin–Shapiro polynomials on the unit circle [Электронный ресурс] / T. Erdélyi. — Режим доступа: <https://arxiv.org/abs/1809.05074>. — 2018.
4. Balister, P. Flat Littlewood Polynomials Exist // P. Balister, B. Bollobás, R. Morris, J. Sahasrabudhe, M. Tiba. *Annals of Mathematics*. — 2019. — DOI: 10.48550/arXiv.1907.09464